SOFTWARE AUTOMATION TEST DOCUMENT

Automatic Smart Pet Feeder — PawFeed Web & Mobile App · Playwright Automation · CI/CD

1. Document Information

Field	Details
Project Name	Automatic Smart Pet Feeder
Application Name	PawFeed — Smart Pet Feeder Web Dashboard & installable Mobile App (PWA), with login and a SQLite database
Version	1.0.0
Test Cycle	Final Exam — Automation Cycle 1
Prepared By	Russel Carlo T. Manzon (IT233)
Reviewed By	
Date	October 17, 2026
Automation Framework	Playwright Test (@playwright/test v1.64.1), JavaScript
Test Environment	Local DEV (Windows 11) · CI (GitHub Actions, Ubuntu) · PROD (Vercel — pawfeed-smart-pet-feeder.vercel.app)
Source Code	github.com/n0zzzy/pawfeed-smart-pet-feeder

2. Objective

Purpose

The objective of this automation testing effort is to turn the automatable scenarios from the *Define Automation Testing* document (AS-01 to AS-06) into real, repeatable Playwright tests that run against the Smart Pet Feeder app on **desktop and on mobile phones (Android and iPhone)**. The tests validate the feeder's key functions — manual feeding, portion accuracy, scheduled dispensing, low-food alerts, Wi-Fi reconnection and app-to-device schedule sync — plus user login, data storage in a SQLite database, and the mobile app itself (installable, offline, responsive). The suite runs automatically in a **CI/CD pipeline** on every code change, and the app is only deployed to production when every test passes, reducing regression effort and capturing screenshot evidence of every run.

3. Scope

In Scope

- Login and authentication (valid and invalid credentials)

- Manual Feed (Feed Now) and jam detection
- Portion size configuration and validation
- Scheduled dispensing using a mocked device clock
- Low Food alert notifications (simulated IR sensor)
- Wi-Fi drop and automatic reconnection (simulated network)
- App-to-device schedule synchronization time
- Data storage in the SQLite database (settings, feed log, user accounts with hashed passwords)
- Mobile app (PWA): installability, offline use, responsive phone layout
- Hosted deployment on Vercel and post-deploy smoke testing of the live site
- Cross-platform execution: Desktop Chrome, Android (Pixel 7) and iPhone 14 (WebKit)

Out of Scope

- Physical / mechanical checks: anti-jam motor force (TC-05), cord safety (TC-11), lid-lock shake test (TC-13) — remain manual / hardware-in-the-loop
- Performance / load testing and penetration testing
- App-store (native) builds and testing on physical phones — phones are emulated by Playwright
- Real feeder firmware / hardware and third-party systems (push-notification providers, Wi-Fi routers)

4. Test Environment

Component	Details
Environment	DEV — local Windows 11 PC · CI — GitHub Actions (ubuntu-latest) · PROD — Vercel
Application URL	Local: http://localhost:3000 (started by Playwright `webServer`) · Live: https://pawfeed-smart-pet-feeder.vercel.app
Database	**SQLite 3.49** (Node.js built-in `node:sqlite`) — app data in `data/pawfeed.db`, tests use a separate `data/pawfeed-test.db`; tables `users`, `device`, `schedules`, `feed_log`, `notifications`. On Vercel (no server) the data stays in the browser's localStorage.
Browser	Chromium (Chrome 156) and WebKit 27.2 (Safari engine), Playwright builds
Mobile Device	Playwright device emulation: **Pixel 7** — Android 14, Chrome, 412×839, touch · **iPhone 14** — iOS Safari (WebKit), 390×664, touch
Operating System	Windows 11 Pro (10.0.26300) locally · Ubuntu (GitHub Actions) in CI
API Endpoint	http://localhost:3000/api (login, status, feed, portion, schedules, sim/*) · on Vercel the device runs inside the app (no server API)
Build Version	PawFeed 1.0.0 · Node.js v23.8.0 (local) / v22 (CI) · Playwright 1.64.1

5. Automation Framework Details

Framework Type

☐ Data Driven

☐ Keyword Driven

☑ **Hybrid Framework** — Playwright Test runner + reusable helper functions (`setPortion`, `lastWeight`, `snap`, `record`), stable `data-testid` locators, test data from the AS-01–AS-06 scenarios, and one suite executed on three platform projects

☐ BDD Framework

☐ Model-Based Testing

Technology Stack

Component	Tool
Test Automation	**Playwright** (@playwright/test 1.64.1)
Mobile Automation	**Playwright device emulation** (Pixel 7, iPhone 14). Appium is not needed because the mobile app is a PWA.
API Automation	Playwright APIRequestContext (device reset, manifest/icon checks)
Programming Language	**JavaScript** (Node.js)
Build Tool	npm (`npm test`, `npm run report`, `npm run allure`, `npm run doc`)
Reporting	**Allure Report** 3 (allure-playwright 3.13) + Playwright HTML Reporter (screenshots, video, trace) + JSON reporter; published live at `/allure` and `/report`
Database	**SQLite** (Node.js `node:sqlite`); passwords hashed with scrypt
CI/CD	**GitHub Actions** → **Vercel** (deploy only after all tests pass)
Source Control	**Git + GitHub** — github.com/n0zzzy/pawfeed-smart-pet-feeder

Project Structure

File / Folder	Purpose
public/index.html	PawFeed app UI — web dashboard and mobile app (system under test)
public/device-core.js	Simulated feeder device logic, shared by the server and the browser
public/manifest.webmanifest, public/sw.js, public/icons/	Mobile app (PWA): install manifest, offline service worker, app icons
server.js	Node.js server + device API used locally and in CI

File / Folder	Purpose
db.js	SQLite database: table schema, scrypt password hashing, load/save of the device data
tests/smart-pet-feeder.spec.js	The 16 automated test cases (ATC-01 – ATC-16)
scripts/allure-report.js	Builds the Allure report from `allure-results/`
playwright.config.js	Runner config: 3 platform projects, base URL, screenshots/video/trace, reporters
.github/workflows/ci-cd.yml	CI/CD pipeline: test → deploy to Vercel → smoke-test the live site
vercel.json	Vercel hosting settings
screenshots/	Screenshots saved by every test (sub-folders for Android and iPhone)

6. Entry Criteria

☑ Stable test environment available (local server starts automatically; CI runner provisioned by GitHub)
☑ Test data prepared (values from AS-01 – AS-06; device reset before each test)
☑ User stories completed (all six scenarios + mobile app features implemented)
☑ Test cases approved (based on the submitted Define Automation Testing document)
☑ Test scripts developed and reviewed
☑ Build deployed successfully (localhost:3000 and Vercel production)

7. Exit Criteria

☑ All critical test cases executed (16 test cases × 3 platforms = 48 executions)
☑ No Sev-1 or Sev-2 defects open
☑ Automation pass rate ≥ 95% (actual: 100%)
☑ Defect retesting completed (all issues in Section 11 re-tested after fix)
☐ Final test report approved (pending instructor review)

8. Automation Test Scenarios

Test ID	Module	Scenario	Maps to	Priority
AT-001	Manual Feed	Verify manual feed dispenses exactly one portion (~10 g)	AS-01 / TC-02	High

Test ID	Module	Scenario	Maps to	Priority
AT-002	Manual Feed	Verify manual feed is blocked when the chute is jammed	AS-01 / TC-02 (negative)	High
AT-003	Portion Settings	Verify 5-slot portion dispenses ~50 g and the setting persists	AS-02 / TC-03	High
AT-004	Portion Settings	Verify an invalid portion (99 slots) is rejected	AS-02 / TC-03 (negative)	High
AT-005	Scheduling	Verify a scheduled meal triggers dispensing on time	AS-03 / TC-04	High
AT-006	Alerts	Verify a Low Food alert is sent when hopper drops below 15%	AS-04 / TC-06	Medium
AT-007	Alerts	Verify no Low Food alert fires above the 15% threshold	AS-04 / TC-06 (negative)	Medium
AT-008	Connectivity	Verify the device auto-reconnects to Wi-Fi without manual setup	AS-05 / TC-08	High
AT-009	App Sync	Verify a schedule change syncs to the device in under 2 seconds	AS-06 / TC-14	Medium
AT-010	Mobile App (PWA)	Verify the mobile app is installable (manifest, icons, service worker)	Mobile app requirement	High
AT-011	Mobile App (PWA)	Verify the app opens and works while the phone is offline	Mobile app requirement	Medium
AT-012	Hosting (Vercel)	Verify the hosted app works without a server API and keeps its data	Deployment requirement	High
AT-013	Responsive UI	Verify the layout fits phone screens with a tappable Feed button	Mobile app requirement	Medium
AT-014	Login & Authentication	Verify Successful Login	Login requirement (template AT-001)	High
AT-015	Login & Authentication	Verify Login with Invalid Credentials	Login requirement (template AT-002)	High
AT-016	Database (SQLite)	Verify settings, feeds and accounts are stored in the database	Data storage requirement	High

Every scenario runs on all three platforms: Desktop Chrome, Android (Pixel 7) and iPhone 14.

9. Automated Test Cases

Each test case below was automated in Playwright and executed on October 17, 2026 at 1:12:53 PM on **Desktop Chrome, Android (Pixel 7) and iPhone 14**. The **Actual Result** values are recorded by the test script itself during the run, and every screenshot was taken automatically by the test at the moment its assertions passed. ATC-10 to ATC-13 show the desktop screenshot next to the top of the Android and iPhone screens.

ATC-01 — Verify manual feed dispenses exactly one portion (~10 g)

Field	Details
Test Case ID	ATC-01
Test Case Name	Verify manual feed dispenses exactly one portion (~10 g)
Module / Maps to	Manual Feed — AS-01 / TC-02
Priority	High
Preconditions	• Feeder is powered on and CONNECTED • Hopper food level is 100% • Portion size is 1 slot (default)

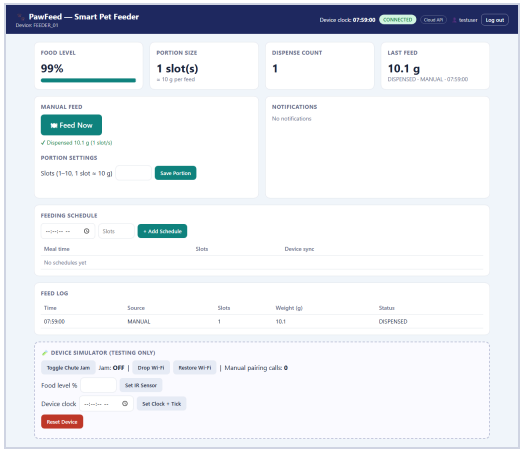
Test Data

Field	Value
Portion size	1 slot (≈10 g)
Tolerance	±5% → 9.5 g – 10.5 g

Test Steps

Step	Description	Expected Result
1	Reset device and open the app	App loads, status CONNECTED, dispense count 0
2	Tap “Feed Now”	Message “Dispensed … g (1 slot/s)” is shown
3	Read Dispense Count and Last Feed weight	Count = 1, weight within 9.5–10.5 g, one row in Feed Log

Field	Details
Expected Result	Exactly 1 dispense event is recorded and the weight is within ±5% of 10 g.
Actual Result	Dispense count went 0 → 1; dispensed 10.1 g (within 9.5–10.5 g). *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.7 s Android: **PASSED** 0.8 s iPhone: **PASSED** 1.3 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-01”



Screenshot: ATC-01_manual_feed_success.png (Desktop Chrome, captured automatically by Playwright)

ATC-02 — Verify manual feed is blocked when the chute is jammed

Field	Details
Test Case ID	ATC-02
Test Case Name	Verify manual feed is blocked when the chute is jammed
Module / Maps to	Manual Feed — AS-01 / TC-02 (negative)
Priority	High
Preconditions	• Feeder is powered on and CONNECTED • Chute jam can be simulated

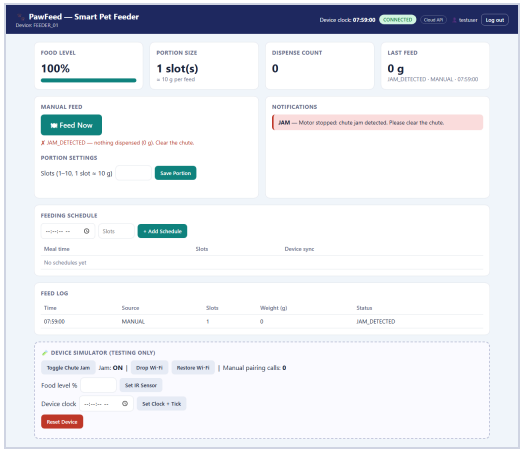
Test Data

Field	Value
Chute jam	ON
Expected status	JAM_DETECTED

Test Steps

Step	Description	Expected Result
1	Tap “Toggle Chute Jam” in the Device Simulator	Jam indicator shows ON
2	Tap “Feed Now”	Message “JAM_DETECTED — nothing dispensed (0 g)”
3	Check Last Feed, Dispense Count and Notifications	Weight 0 g, count stays 0, JAM notification visible

Field	Details
Expected Result	Feeder reports JAM_DETECTED, dispenses 0 g and alerts the owner.
Actual Result	Status JAM_DETECTED, 0 g dispensed, dispense count stayed 0, JAM notification shown. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.7 s Android: **PASSED** 1.1 s iPhone: **PASSED** 1.3 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-02”



Screenshot: ATC-02_manual_feed_jam_detected.png (Desktop Chrome, captured automatically by Playwright)

ATC-03 — Verify 5-slot portion dispenses ~50 g and the setting persists

Field	Details
Test Case ID	ATC-03
Test Case Name	Verify 5-slot portion dispenses ~50 g and the setting persists
Module / Maps to	Portion Settings — AS-02 / TC-03
Priority	High
Preconditions	• App is open and paired with the device

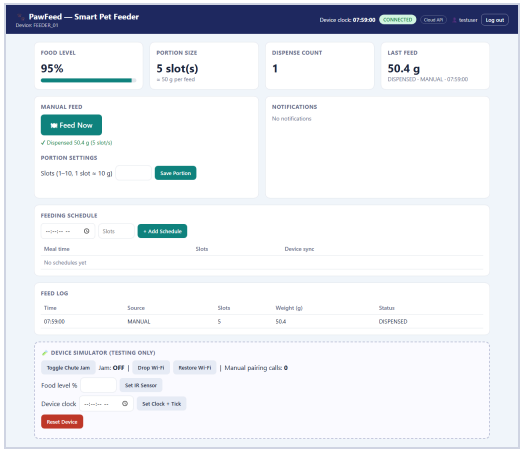
Test Data

Field	Value
Portion size	5 slots (≈50 g)
Tolerance	±5% → 47.5 g – 52.5 g

Test Steps

Step	Description	Expected Result
1	Enter 5 in the Slots field and tap “Save Portion”	“Portion saved: 5 slot(s)” message
2	Reload the app	Portion Size still shows 5 slot(s)
3	Tap “Feed Now” and read Last Feed weight	Weight within 47.5–52.5 g

Field	Details
Expected Result	Portion is saved on the device and the dispensed weight is within ±5% of 50 g.
Actual Result	Portion saved and still 5 slots after page reload; dispensed 50.4 g (within 47.5–52.5 g). *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.8 s Android: **PASSED** 0.9 s iPhone: **PASSED** 1.5 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-03”



Screenshot: ATC-03_portion_5_slots_50g.png (Desktop Chrome, captured automatically by Playwright)

ATC-04 — Verify an invalid portion (99 slots) is rejected

Field	Details
Test Case ID	ATC-04
Test Case Name	Verify an invalid portion (99 slots) is rejected
Module / Maps to	Portion Settings — AS-02 / TC-03 (negative)
Priority	High
Preconditions	• App is open and paired with the device • Current portion is 1 slot

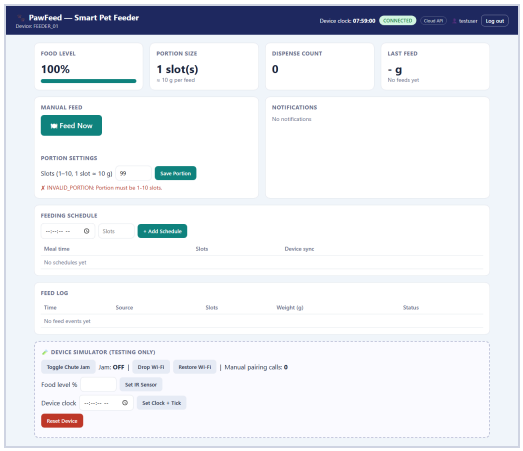
Test Data

Field	Value
Portion size	99 slots (exceeds 1–10 limit)
Expected status	INVALID_PORTION

Test Steps

Step	Description	Expected Result
1	Enter 99 in the Slots field and tap “Save Portion”	Error “INVALID_PORTION: Portion must be 1-10 slots.”
2	Check Portion Size and Dispense Count	Portion stays 1 slot, nothing dispensed

Field	Details
Expected Result	The value is rejected with INVALID_PORTION and is not saved or dispensed.
Actual Result	App showed "INVALID_PORTION: Portion must be 1-10 slots."; portion stayed at 1 slot. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.7 s Android: **PASSED** 1.0 s iPhone: **PASSED** 1.2 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-04”



Screenshot: ATC-04_invalid_portion_rejected.png (Desktop Chrome, captured automatically by Playwright)

ATC-05 — Verify a scheduled meal triggers dispensing on time

Field	Details
Test Case ID	ATC-05
Test Case Name	Verify a scheduled meal triggers dispensing on time
Module / Maps to	Scheduling — AS-03 / TC-04
Priority	High
Preconditions	• Device clock is synced (starts at 07:59:00) • Schedule can be created in the app

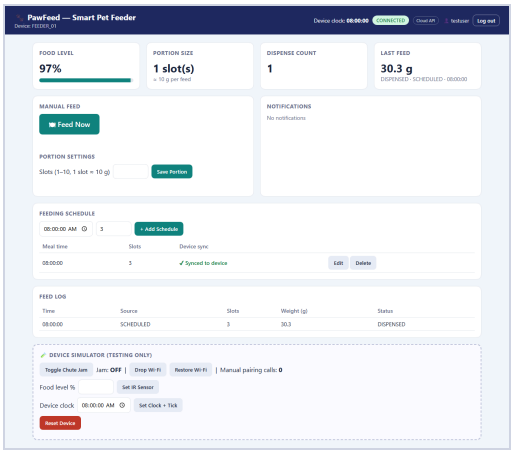
Test Data

Field	Value
Meal time	08:00:00
Portion	3 slots (≈30 g)
Time window	±1 minute

Test Steps

Step	Description	Expected Result
1	Add a schedule for 08:00:00 with 3 slots	Row appears and shows “Synced to device”
2	Set the device clock to 08:00:00 and tick (mocked clock)	Device clock shows 08:00:00
3	Check the Feed Log	New SCHEDULED / DISPENSED row within ±60 s of 08:00:00, ≈30 g

Field	Details
Expected Result	A scheduled feed event is logged within ±1 minute of the scheduled time.
Actual Result	SCHEDULED feed logged at 08:00:00 (0 s from target, within ±60 s); dispensed 30.3 g for 3 slots. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 1.6 s Android: **PASSED** 1.8 s iPhone: **PASSED** 2.4 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-05”



Screenshot: ATC-05_scheduled_feed_triggered.png (Desktop Chrome, captured automatically by Playwright)

ATC-06 — Verify a Low Food alert is sent when hopper drops below 15%

Field	Details
Test Case ID	ATC-06
Test Case Name	Verify a Low Food alert is sent when hopper drops below 15%
Module / Maps to	Alerts — AS-04 / TC-06
Priority	Medium
Preconditions	• IR sensor reading can be simulated

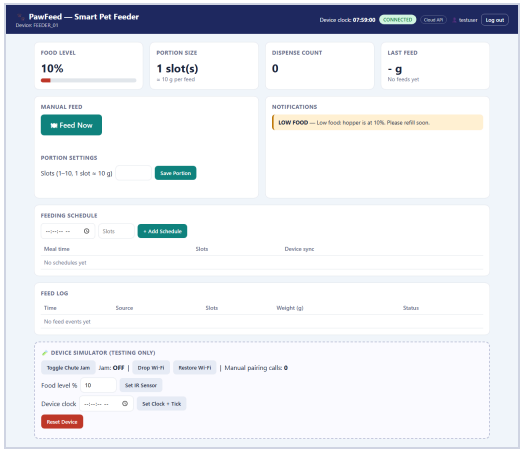
Test Data

Field	Value
Food level	10%
Alert threshold	< 15%

Test Steps

Step	Description	Expected Result
1	Enter 10 in “Food level %” and tap “Set IR Sensor”	Food Level shows 10% (red bar)
2	Watch the Notifications panel (max 3 s)	“LOW FOOD” notification appears

Field	Details
Expected Result	The app receives a Low Food notification within a few seconds.
Actual Result	Food level showed 10% (red bar) and a LOW FOOD notification appeared. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.7 s Android: **PASSED** 0.8 s iPhone: **PASSED** 1.9 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-06”



Screenshot: ATC-06_low_food_alert_10_percent.png (Desktop Chrome, captured automatically by Playwright)

ATC-07 — Verify no Low Food alert fires above the 15% threshold

Field	Details
Test Case ID	ATC-07
Test Case Name	Verify no Low Food alert fires above the 15% threshold
Module / Maps to	Alerts — AS-04 / TC-06 (negative)
Priority	Medium
Preconditions	• IR sensor reading can be simulated

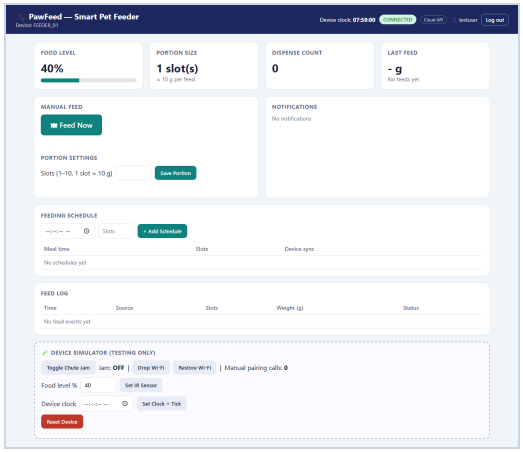
Test Data

Field	Value
Food level	40%
Alert threshold	< 15%

Test Steps

Step	Description	Expected Result
1	Enter 40 in “Food level %” and tap “Set IR Sensor”	Food Level shows 40%
2	Wait 1.5 s and check Notifications	No LOW FOOD notification

Field	Details
Expected Result	No Low Food alert is shown when the hopper is above 15%.
Actual Result	Food level showed 40%; no LOW FOOD notification after waiting 1.5 s. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 2.2 s Android: **PASSED** 2.4 s iPhone: **PASSED** 2.9 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-07”



Screenshot: ATC-07_no_alert_40_percent.png (Desktop Chrome, captured automatically by Playwright)

ATC-08 — Verify the device auto-reconnects to Wi-Fi without manual setup

Field	Details
Test Case ID	ATC-08
Test Case Name	Verify the device auto-reconnects to Wi-Fi without manual setup
Module / Maps to	Connectivity — AS-05 / TC-08
Priority	High
Preconditions	• Device was previously paired and CONNECTED

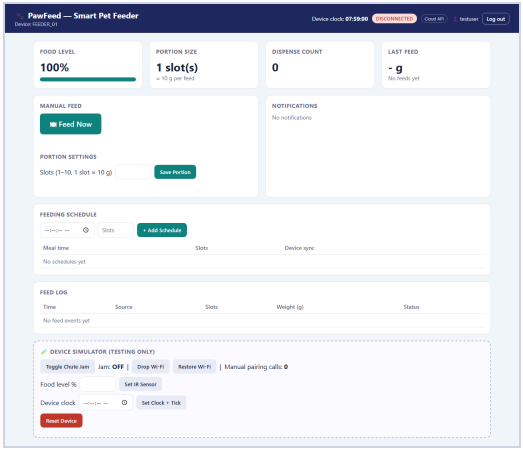
Test Data

Field	Value
Outage length	5 s
Reconnect timeout	60 s
Allowed pairing calls	0

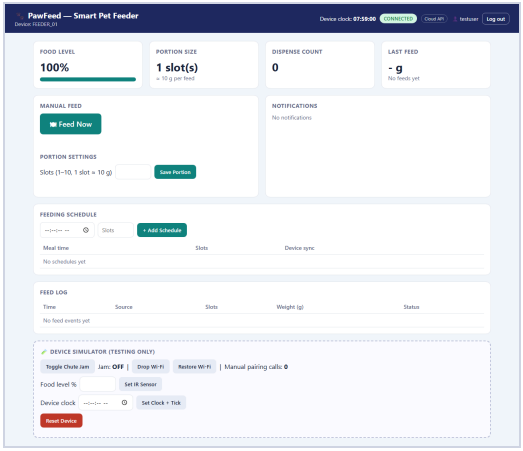
Test Steps

Step	Description	Expected Result
1	Tap “Drop Wi-Fi”	Status badge shows DISCONNECTED
2	Wait 5 s, then tap “Restore Wi-Fi”	Status shows RECONNECTING
3	Poll the status badge (up to 60 s)	Status returns to CONNECTED
4	Check “Manual pairing calls”	Value is 0

Field	Details
Expected Result	Status returns to CONNECTED within 60 s with no manual pairing call.
Actual Result	Status went DISCONNECTED → RECONNECTING → CONNECTED in 3.4 s after Wi-Fi was restored; manual pairing calls = 0. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 9.2 s Android: **PASSED** 9.4 s iPhone: **PASSED** 10.2 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-08”



Screenshot: ATC-08a_wifi_dropped.png (Desktop Chrome, captured automatically by Playwright)



Screenshot: ATC-08b_wifi_reconnected.png (Desktop Chrome, captured automatically by Playwright)

ATC-09 — Verify a schedule change syncs to the device in under 2 seconds

Field	Details
Test Case ID	ATC-09
Test Case Name	Verify a schedule change syncs to the device in under 2 seconds
Module / Maps to	App Sync — AS-06 / TC-14
Priority	Medium
Preconditions	• App and device are online and paired • One schedule exists (08:00:00)

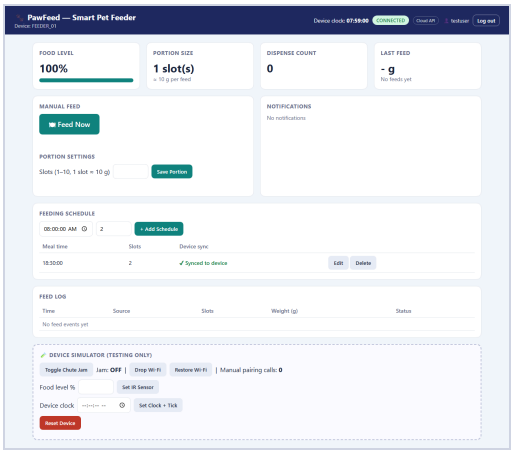
Test Data

Field	Value
Original time	08:00:00
New time	18:30:00
Sync limit	< 2 s

Test Steps

Step	Description	Expected Result
1	Add a schedule at 08:00:00 and wait for “Synced to device”	Schedule synced
2	Tap “Edit”, change the time to 18:30:00 and start the timer on “Save”	Row shows 18:30:00, “Syncing…”
3	Wait for “Synced to device” and stop the timer	Elapsed time under 2 s

Field	Details
Expected Result	The device schedule matches the app’s new value in under 2 seconds.
Actual Result	• **Desktop:** Schedule changed 08:00:00 → 18:30:00 and showed "Synced to device" after 0.82 s (limit < 2 s). • **Android:** Schedule changed 08:00:00 → 18:30:00 and showed "Synced to device" after 1.14 s (limit < 2 s). • **iPhone:** Schedule changed 08:00:00 → 18:30:00 and showed "Synced to device" after 0.68 s (limit < 2 s).
Execution Result	Desktop: **PASSED** 2.3 s Android: **PASSED** 2.7 s iPhone: **PASSED** 4.0 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-09”



Screenshot: ATC-09_schedule_sync_under_2s.png (Desktop Chrome, captured automatically by Playwright)

ATC-10 — Verify the mobile app is installable (manifest, icons, service worker)

Field	Details
Test Case ID	ATC-10
Test Case Name	Verify the mobile app is installable (manifest, icons, service worker)
Module / Maps to	Mobile App (PWA) — Mobile app requirement
Priority	High
Preconditions	• App is served over localhost or HTTPS (Vercel)

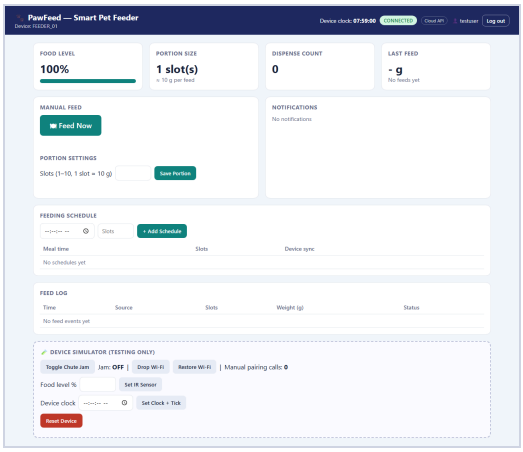
Test Data

Field	Value
Manifest	/manifest.webmanifest
Required	display: standalone, start_url: /, icons 192 & 512 px
Service worker	/sw.js

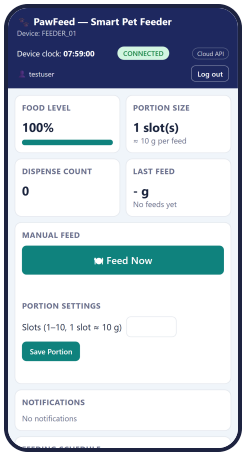
Test Steps

Step	Description	Expected Result
1	Open the app	Page links /manifest.webmanifest
2	Request the manifest	name PawFeed, display standalone, start_url /
3	Request every icon in the manifest	All icons load (HTTP 200)
4	Check the service worker registration	Service worker state is “activated”

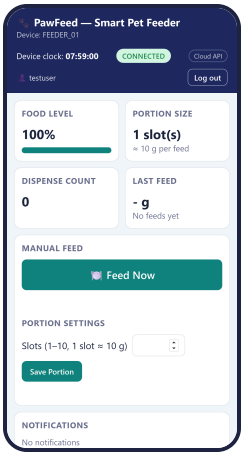
Field	Details
Expected Result	The phone’s browser can install PawFeed as an app (Add to Home Screen).
Actual Result	Manifest "PawFeed — Smart Pet Feeder" (display: standalone) with 3 icons loaded; service worker is activated. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.6 s Android: **PASSED** 0.7 s iPhone: **PASSED** 1.5 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-10” · tag @smoke (also run against the live site)



Desktop Chrome



Android · Pixel 7 (Chrome)



iPhone 14 (Safari / WebKit)

ATC-11 — Verify the app opens and works while the phone is offline

Field	Details
Test Case ID	ATC-11
Test Case Name	Verify the app opens and works while the phone is offline
Module / Maps to	Mobile App (PWA) — Mobile app requirement
Priority	Medium
Preconditions	• App was opened once, so the service worker cached it

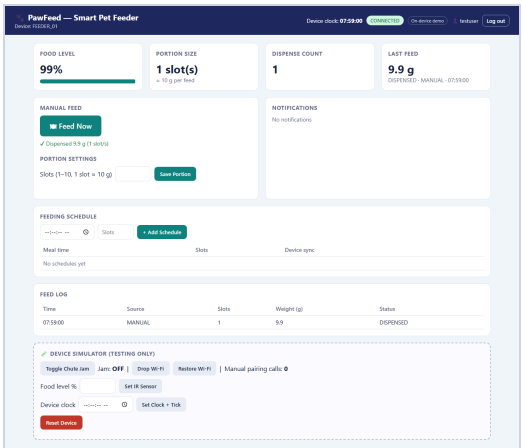
Test Data

Field	Value
Network	Offline
Expected mode	On-device demo

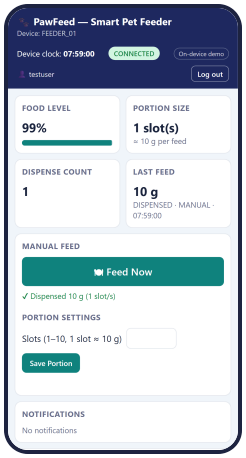
Test Steps

Step	Description	Expected Result
1	Wait for the service worker to be activated	State “activated”
2	Turn the network off and reload the app	App still opens (served from cache)
3	Check the mode label	“On-device demo”
4	Tap “Feed Now”	“Dispensed … g” message

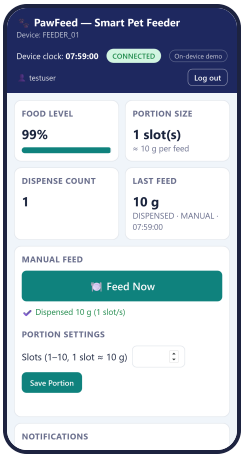
Field	Details
Expected Result	The app loads and can feed without an internet connection.
Actual Result	With the network turned off, the app loaded from the service-worker cache, switched to on-device mode and dispensed a portion. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.6 s Android: **PASSED** 1.1 s iPhone: **PASSED** 2.0 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-11” · tag @smoke (also run against the live site)



Desktop Chrome



Android · Pixel 7 (Chrome)



iPhone 14 (Safari / WebKit)

ATC-12 — Verify the hosted app works without a server API and keeps its data

Field	Details
Test Case ID	ATC-12
Test Case Name	Verify the hosted app works without a server API and keeps its data
Module / Maps to	Hosting (Vercel) — Deployment requirement
Priority	High
Preconditions	• No /api server is available (same as on Vercel)

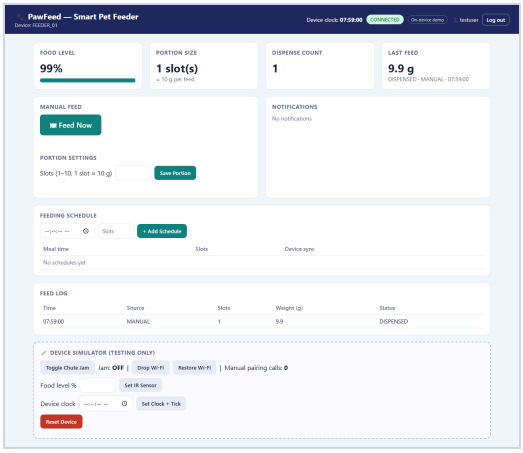
Test Data

Field	Value
/api/** response	404 Not Found (mocked)
Storage	Browser localStorage

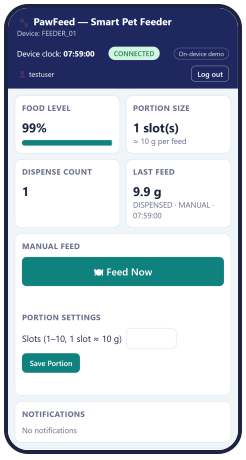
Test Steps

Step	Description	Expected Result
1	Open the app	Mode label shows “On-device demo”, status CONNECTED
2	Tap “Feed Now”	Dispense count 1, weight 9.5–10.5 g
3	Reload the app	Dispense count is still 1

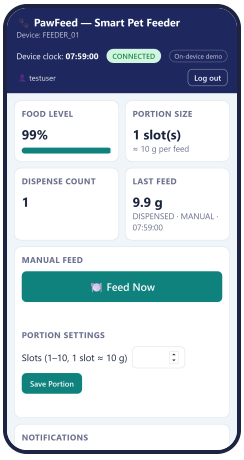
Field	Details
Expected Result	The Vercel-hosted app runs the feeder in the browser and keeps the data after reload.
Actual Result	With /api unavailable (like on Vercel) the app ran in on-device mode, dispensed 9.9 g and still showed 1 feed after reload. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.7 s Android: **PASSED** 1.4 s iPhone: **PASSED** 1.5 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-12” · tag @smoke (also run against the live site)



Desktop Chrome



Android · Pixel 7 (Chrome)



iPhone 14 (Safari / WebKit)

ATC-13 — Verify the layout fits phone screens with a tappable Feed button

Field	Details
Test Case ID	ATC-13
Test Case Name	Verify the layout fits phone screens with a tappable Feed button
Module / Maps to	Responsive UI — Mobile app requirement
Priority	Medium
Preconditions	• App is open on the target screen size

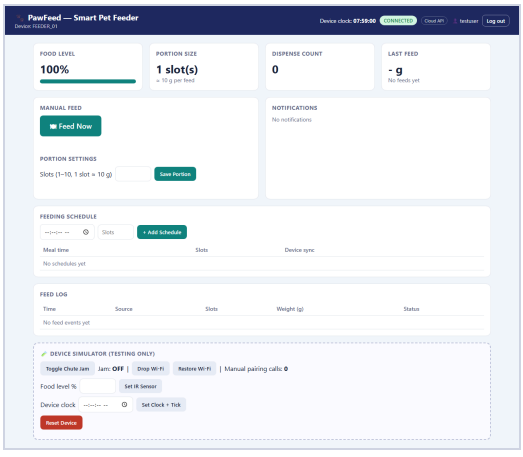
Test Data

Field	Value
Screens	1280×900 · 412×839 (Pixel 7) · 390×664 (iPhone 14)
Min. tap target	44 × 44 px

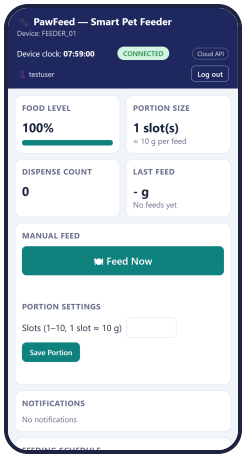
Test Steps

Step	Description	Expected Result
1	Compare page width with screen width	No horizontal (sideways) scrolling
2	Check the “Feed Now” button	Visible on the first screen
3	Measure the button	At least 44 × 44 px

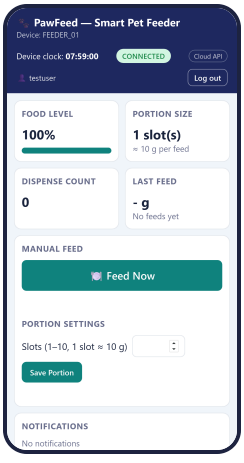
Field	Details
Expected Result	The app fits every screen and the main action is easy to tap.
Actual Result	• **Desktop:** Viewport 1280×900: page width 1280 px fits 1280 px; Feed Now button 153×52 px and visible. • **Android:** Viewport 412×839: page width 412 px fits 412 px; Feed Now button 362×52 px and visible. • **iPhone:** Viewport 390×664: page width 390 px fits 390 px; Feed Now button 340×52 px and visible.
Execution Result	Desktop: **PASSED** 0.6 s Android: **PASSED** 1.1 s iPhone: **PASSED** 1.2 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-13” · tag @smoke (also run against the live site)



Desktop Chrome



Android · Pixel 7 (Chrome)



iPhone 14 (Safari / WebKit)

ATC-14 — Verify Successful Login

Field	Details
Test Case ID	ATC-14
Test Case Name	Verify Successful Login
Module / Maps to	Login & Authentication — Login requirement (template AT-001)
Priority	High
Preconditions	• User account exists (stored in the SQLite users table) • Application available

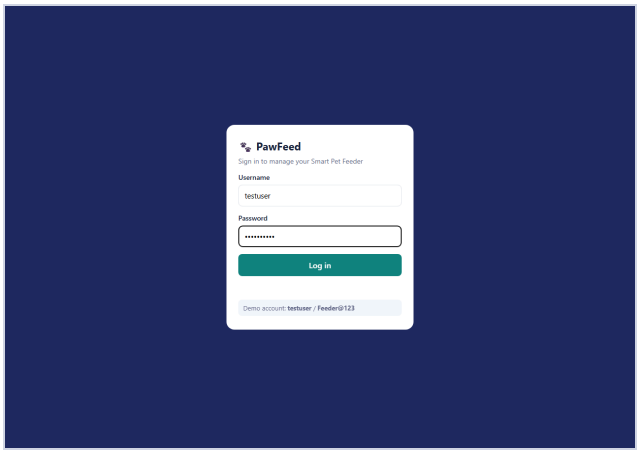
Test Data

Field	Value
Username	testuser
Password	********

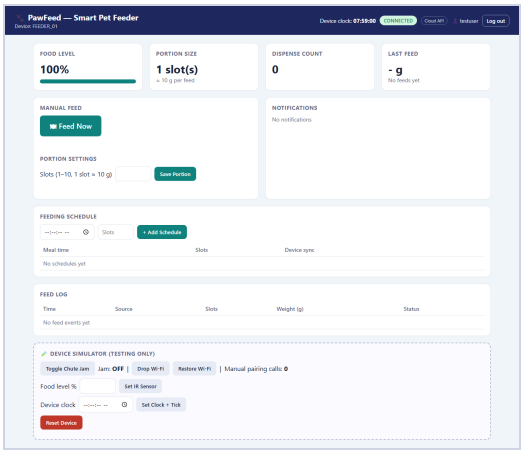
Test Steps

Step	Description	Expected Result
1	Launch Application	Login page displayed
2	Enter Username	Username accepted
3	Enter Password	Password accepted (masked)
4	Click Login	Dashboard displayed

Field	Details
Expected Result	User successfully logs in and dashboard is displayed.
Actual Result	Login page shown; username "testuser" and masked password accepted; after Log in the dashboard opened showing " testuser". *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.5 s Android: **PASSED** 1.0 s iPhone: **PASSED** 1.2 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-14”



Screenshot: ATC-14a_login_page.png (Desktop Chrome, captured automatically by Playwright)



Screenshot: ATC-14b_dashboard_displayed.png (Desktop Chrome, captured automatically by Playwright)

ATC-15 — Verify Login with Invalid Credentials

Field	Details
Test Case ID	ATC-15
Test Case Name	Verify Login with Invalid Credentials
Module / Maps to	Login & Authentication — Login requirement (template AT-002)
Priority	High
Preconditions	• User account exists • Application available

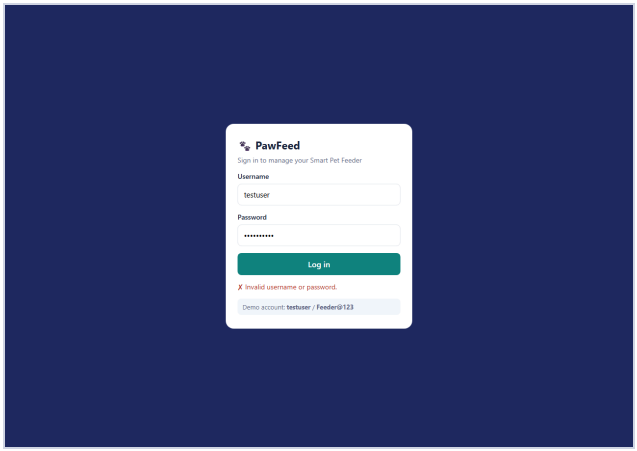
Test Data

Field	Value
Username	testuser
Password	******** (wrong password)

Test Steps

Step	Description	Expected Result
1	Launch Application	Login page displayed
2	Enter Username and a wrong Password	Values accepted
3	Click Login	Error “Invalid username or password.” is shown
4	Check the screen	Still on the login page; dashboard not displayed

Field	Details
Expected Result	Login is rejected with an error message and the dashboard is not displayed.
Actual Result	App showed "✗ Invalid username or password." and stayed on the login page; the dashboard was not shown. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.4 s Android: **PASSED** 0.5 s iPhone: **PASSED** 0.8 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-15”



Screenshot: ATC-15_invalid_login_rejected.png (Desktop Chrome, captured automatically by Playwright)

ATC-16 — Verify settings, feeds and accounts are stored in the database

Field	Details
Test Case ID	ATC-16
Test Case Name	Verify settings, feeds and accounts are stored in the database
Module / Maps to	Database (SQLite) — Data storage requirement
Priority	High
Preconditions	• Server is running with the SQLite test database (data/pawfeed-test.db) • User is logged in

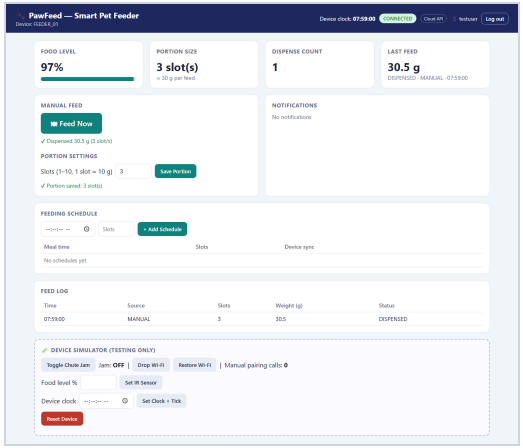
Test Data

Field	Value
Database	SQLite — tables users, device, schedules, feed_log, notifications
Portion size	3 slots
Check method	Test opens the database file read-only and runs SQL queries

Test Steps

Step	Description	Expected Result
1	Save a portion of 3 slots and tap “Feed Now”	Portion 3, dispense count 1
2	Query the device table	portion_slots = 3, dispense_count = 1
3	Query the newest feed_log row	DISPENSED, 3 slots, same weight as shown in the app
4	Query the users table for testuser	Password stored as a scrypt hash, not plain text

Field	Details
Expected Result	App data is written to the SQLite database and passwords are stored hashed.
Actual Result	device row: portion_slots=3, dispense_count=1; feed_log row: MANUAL 3 slots 30.5 g DISPENSED (matches the app); users row stores a scrypt hash, not the plain password. *(Desktop run shown; Android and iPhone runs met the same limits.)*
Execution Result	Desktop: **PASSED** 0.8 s Android: **PASSED** 0.9 s iPhone: **PASSED** 1.2 s
Automation Status	☐ Not Started ☐ In Progress ☑ Automated ☑ Executed
Script	tests/smart-pet-feeder.spec.js — “ATC-16”



Screenshot: ATC-16_saved_in_database.png (Desktop Chrome, captured automatically by Playwright)

10. Test Data Management

Data Type	Source	Owner
User Accounts	SQLite `users` table — demo account `testuser`, created when the database is first opened; password stored as a scrypt hash	QA Team
Device state (food level, portion, schedules, logs)	Reset through `POST /api/sim/reset` in `beforeEach`	QA (Playwright hook)
Portion sizes, meal times, thresholds	Hard-coded in the spec from AS-01 – AS-06	QA Team
Sensor & hardware inputs (jam, IR food level, Wi-Fi, clock)	Device Simulator panel / `/api/sim/*`	QA Team
On-device data (Vercel / offline mode)	Browser localStorage; each test uses a fresh browser context	QA (Playwright)
Deployment secrets (VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID)	GitHub Actions encrypted secrets — never stored in the code	Russel Carlo T. Manzon

11. Defect Tracking

No open defects: all 48 test executions passed in the final run. Issues found while building and running the automation were fixed and re-tested:

Defect ID	Summary	Severity	Status
BUG-001	On iPhone (WebKit) the “Feed Now” and other buttons used a serif system font instead of the app font. Found from the iPhone screenshots; fixed by making buttons and inputs inherit the page font.	Low	Fixed
ENV-001	Playwright took ~3 minutes to stop the app server on Windows after each run. Fixed by adding a global teardown that asks the server to shut down.	Low	Fixed
ENV-002	Food-level bar was captured mid-animation, so the screenshot showed a shorter bar than the % value. Fixed by removing the CSS transition.	Low	Fixed
ENV-003	Vercel cannot keep the server’s in-memory device state between requests. Fixed by moving the device logic into a shared module that also runs inside the app (covered by ATC-12).	Medium	Fixed

Defect ID	Summary	Severity	Status
ENV-004	ATC-16 could not open the database because Playwright's test-file loader cannot `require('node:sqlite')`. Fixed by loading it with `process.getBuiltinModule()`.	Low	Fixed
ENV-005	On Windows, Allure report generation sometimes failed with EPERM while renaming a freshly written folder (file briefly locked). Fixed with a script that retries the generation (`npm run allure`).	Low	Fixed

Severity Classification: Critical · High · Medium · Low

12. Test Execution Summary

Metric	Value
Total Test Cases	**16**
Automated Test Cases	**16**
Platforms	**3 (Desktop, Android, iPhone)**
Executed Test Runs	**48 (16 × 3)**
Passed	**48**
Failed	**0**
Blocked / Skipped	**0**
Flaky	**0**
Pass Percentage	**100%**
Total Execution Time	**93.1 s**
Run Date / Time	**October 17, 2026 at 1:12:53 PM**

Formula: Pass Rate (%) = (Passed / Executed) × 100 = (48 / 48) × 100 = **100%**

Results per Platform

Test Case	Desktop Chrome	Android · Pixel 7 (Chrome)	iPhone 14 (Safari / WebKit)
ATC-01 — Manual Feed	✓ Passed	✓ Passed	✓ Passed
ATC-02 — Manual Feed	✓ Passed	✓ Passed	✓ Passed
ATC-03 — Portion Settings	✓ Passed	✓ Passed	✓ Passed

Test Case	Desktop Chrome	Android · Pixel 7 (Chrome)	iPhone 14 (Safari / WebKit)
ATC-04 — Portion Settings	✓ Passed	✓ Passed	✓ Passed
ATC-05 — Scheduling	✓ Passed	✓ Passed	✓ Passed
ATC-06 — Alerts	✓ Passed	✓ Passed	✓ Passed
ATC-07 — Alerts	✓ Passed	✓ Passed	✓ Passed
ATC-08 — Connectivity	✓ Passed	✓ Passed	✓ Passed
ATC-09 — App Sync	✓ Passed	✓ Passed	✓ Passed
ATC-10 — Mobile App (PWA)	✓ Passed	✓ Passed	✓ Passed
ATC-11 — Mobile App (PWA)	✓ Passed	✓ Passed	✓ Passed
ATC-12 — Hosting (Vercel)	✓ Passed	✓ Passed	✓ Passed
ATC-13 — Responsive UI	✓ Passed	✓ Passed	✓ Passed
ATC-14 — Login & Authentication	✓ Passed	✓ Passed	✓ Passed
ATC-15 — Login & Authentication	✓ Passed	✓ Passed	✓ Passed
ATC-16 — Database (SQLite)	✓ Passed	✓ Passed	✓ Passed

Execution Evidence

Screenshots taken by the tester during the run on October 17, 2026 at 1:12:53 PM: every test case passed on every platform. The same suite also passed in the CI/CD pipeline (Section 16), and the Allure report of this run is shown in Section 15.

[Tester screenshots of this run not added yet]

13. Automation Coverage

Module	Total TC	Automated	Coverage %
Manual Feed (TC-02)	2	2	100%
Portion Settings (TC-03)	2	2	100%
Scheduling (TC-04)	1	1	100%
Low Food Alerts (TC-06)	2	2	100%

Module	Total TC	Automated	Coverage %
Wi-Fi Connectivity (TC-08)	1	1	100%
App Schedule Sync (TC-14)	1	1	100%
Mobile App — PWA install & offline	2	2	100%
Hosting (Vercel on-device mode)	1	1	100%
Responsive UI (phone layouts)	1	1	100%
Login & Authentication	2	2	100%
Database (SQLite)	1	1	100%
Physical / Hardware (TC-05, TC-11, TC-13)	3	0	0% (manual only)
Total	**19**	**16**	**84%**

> ATC-02 covers the software side of TC-05 (jam is detected and the motor stops). The physical anti-jam force, cord integrity and lid-lock shake test need real hardware, so they stay manual as stated in the Define Automation Testing document. Platform coverage is 100%: every automated test case runs on desktop, Android and iPhone.

14. Risks and Assumptions

Risks

- The feeder device is simulated; real hardware may behave differently (motor timing, sensor noise).
- Phones are emulated (screen size, touch, user agent, browser engine); a real phone may still differ slightly.
- Timing-based tests (Wi-Fi reconnect, sync < 2 s) could become flaky on a slow CI runner.
- Dispensed weight has random ±2% variance, so weight checks rely on the ±5% tolerance.
- Deployment depends on GitHub Actions and Vercel being available, and on the Vercel token staying valid.

Assumptions

- Port 3000 is free and Node.js + Playwright Chromium and WebKit are installed.
- The requirements in the Test Plan / Define Automation Testing document are final.
- Tests run one at a time (1 worker) because they share one simulated device.

15. Reporting and Dashboard

Daily Metrics

- Build Status: shown on every commit in GitHub Actions (github.com/n0zzzy/pawfeed-smart-pet-feeder/actions)
- Execution Status: 48/48 passed

- Defect Trend: 0 open; 6 fixed (1 UI bug, 5 environment issues)
- Automation Coverage: 16 of 19 test cases (84%), on 3 platforms
- Failed Tests: 0
- Execution Duration: 93.1 s locally

Recommended Reports

Recommended Report	Used	How it is covered in this project
Allure Report	☑	**Allure Report 3** through the official `allure-playwright` reporter: summary, every test with its steps, screenshots, video and trace. Built with `npm run allure`, published by the pipeline at pawfeed-smart-pet-feeder.vercel.app/allure.
Extent Report	☐	Not used — Allure and the Playwright HTML report already give the same detail. Extent Reports is mainly used with Java/.NET Selenium frameworks (TestNG, NUnit).
Azure DevOps Dashboard	☐	Covered by **GitHub Actions** — the code is on GitHub, so build status, run history, the README status badge and downloadable artifacts (`playwright-report`, `allure-report`, `screenshots`) are shown there.
Power BI Test Dashboard	☐	Covered by the **Playwright JSON report** (`docs/results.json`) and the per-platform results table in Section 12. The same JSON file can be imported into Power BI later.

Also kept from earlier: the **Playwright HTML Report** (`npm run report`, live at /report), **GitHub Actions** run summaries and the **JSON report**.

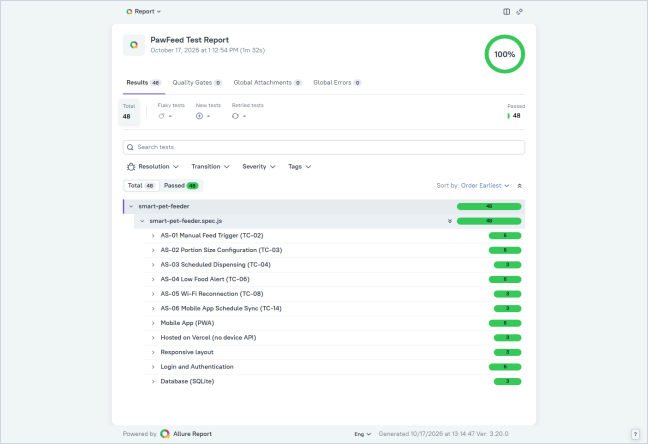


Figure 2 — Allure Report: 48 of 48 passed (100%), grouped by test suite

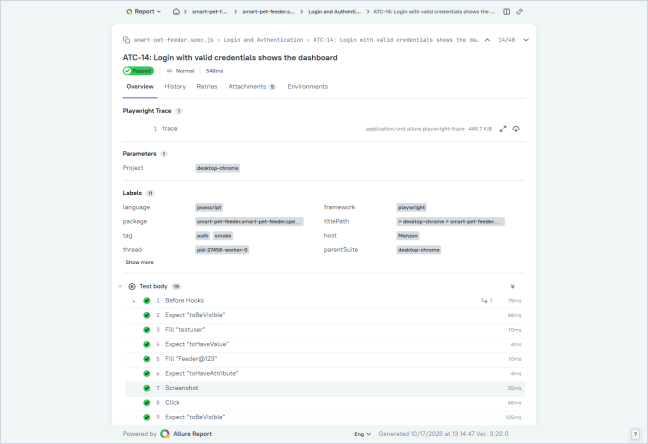


Figure 3 — Allure Report: ATC-14 “Verify Successful Login” with its test steps

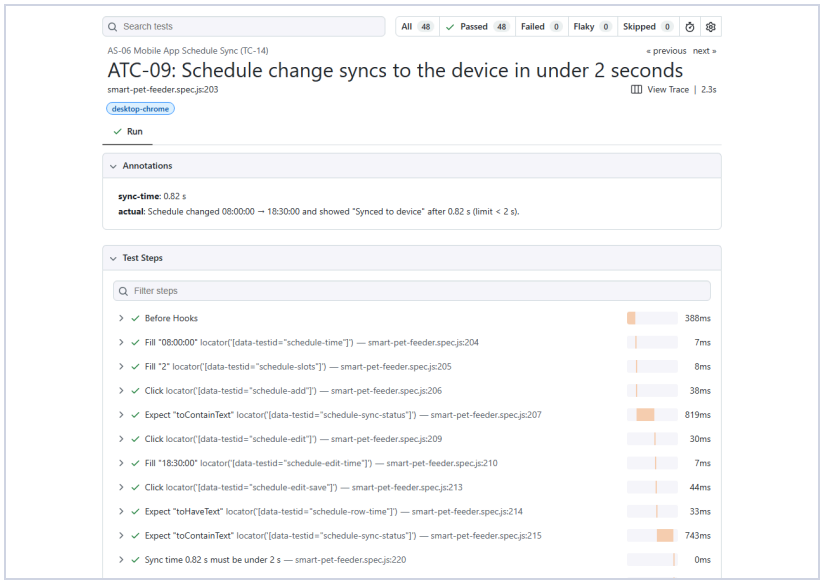


Figure 4 — Playwright HTML report — step-by-step detail of ATC-09 with the recorded sync time

16. CI/CD Integration Checklist

1. Push / Pull Request
Code pushed to GitHub (main)

→

2. Test
16 test cases × 3 platforms on Ubuntu

→

3. Quality gate
Deploy job runs only if every test passed

→

4. Deploy
App + Playwright & Allure reports to Vercel

→

5. Smoke test
@smoke tests against the live URL

Item	Status	Notes
Source Code in Git	☑	GitHub: github.com/n0zzzy/pawfeed-smart-pet-feeder
Build Pipeline Configured	☑	GitHub Actions workflow `.github/workflows/ci-cd.yml`
Automated Execution Triggered	☑	Runs on every push and pull request to `main` (and manually)
Test Reports Published	☑	Playwright HTML report, Allure report and screenshots as run artifacts, and live at /report and /allure
Notifications Enabled	☑	GitHub emails the repository owner when a workflow run fails
Quality Gates Configured	☑	Deploy needs the test job to pass; the live site is then smoke-tested

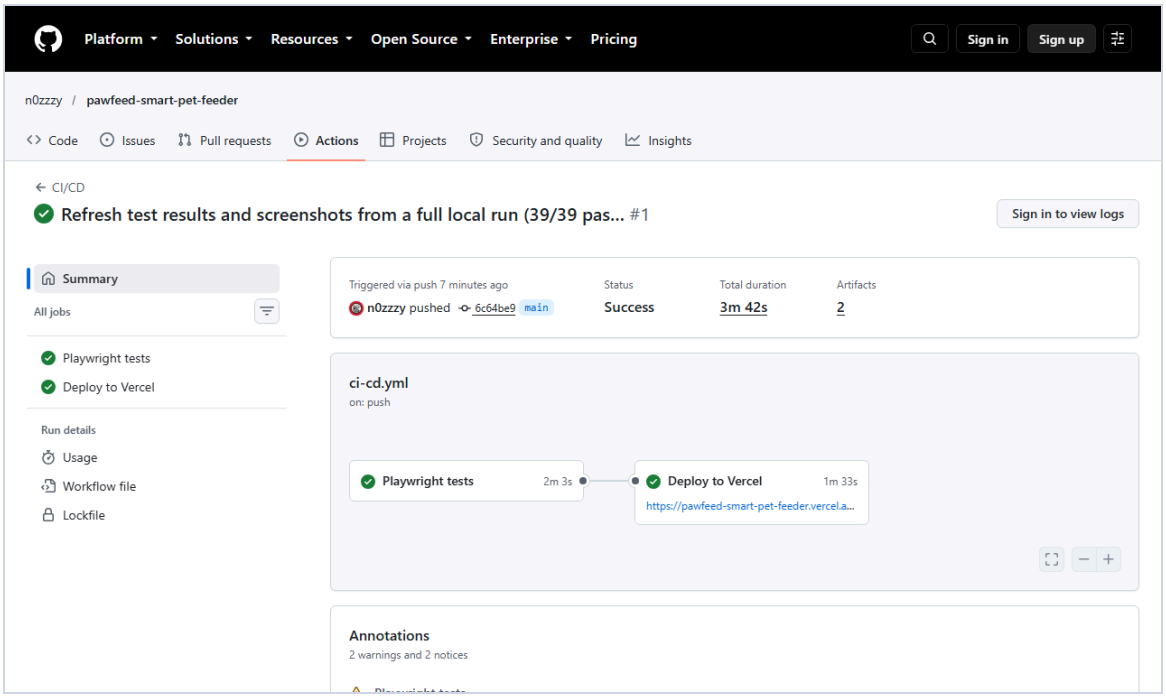


Figure 5 — GitHub Actions: CI/CD run succeeded — Playwright tests → Deploy to Vercel (with live smoke test)

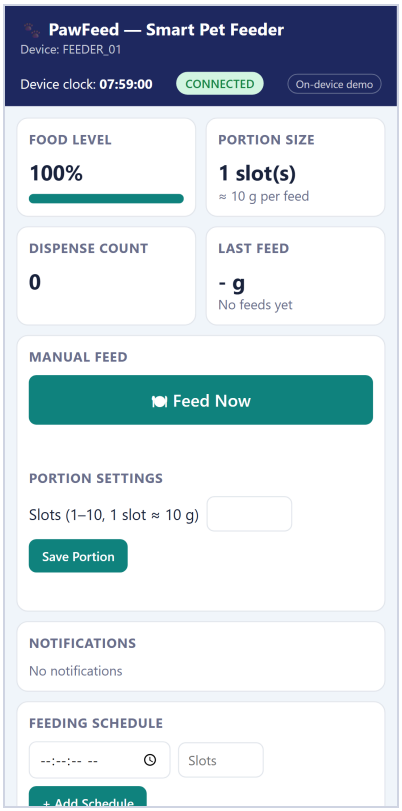


Figure 6 — The live app on Vercel (pawfeed-smart-pet-feeder.vercel.app) on an emulated Android phone

17. Test Sign-Off

Role	Name	Signature	Date
QA Lead	Russel Carlo T. Manzon		October 17, 2026
Test Manager	Russel Carlo T. Manzon		October 17, 2026
Development Lead	Russel Carlo T. Manzon		October 17, 2026
Business Owner	Russel Carlo T. Manzon		October 17, 2026

Solo project: every role was carried out by the student, Russel Carlo T. Manzon. Instructor review is recorded under “Reviewed By” in Section 1.

Final Recommendation

Go Live Status:

☐ Go

☑ **Conditional Go** — all 48 automated test runs passed (100%) on desktop, Android and iPhone, and the app is live on Vercel. Go-live for the physical feeder is conditional on completing the manual hardware tests (TC-05 anti-jam force, TC-11 cord safety, TC-13 lid lock).

☐ No-Go